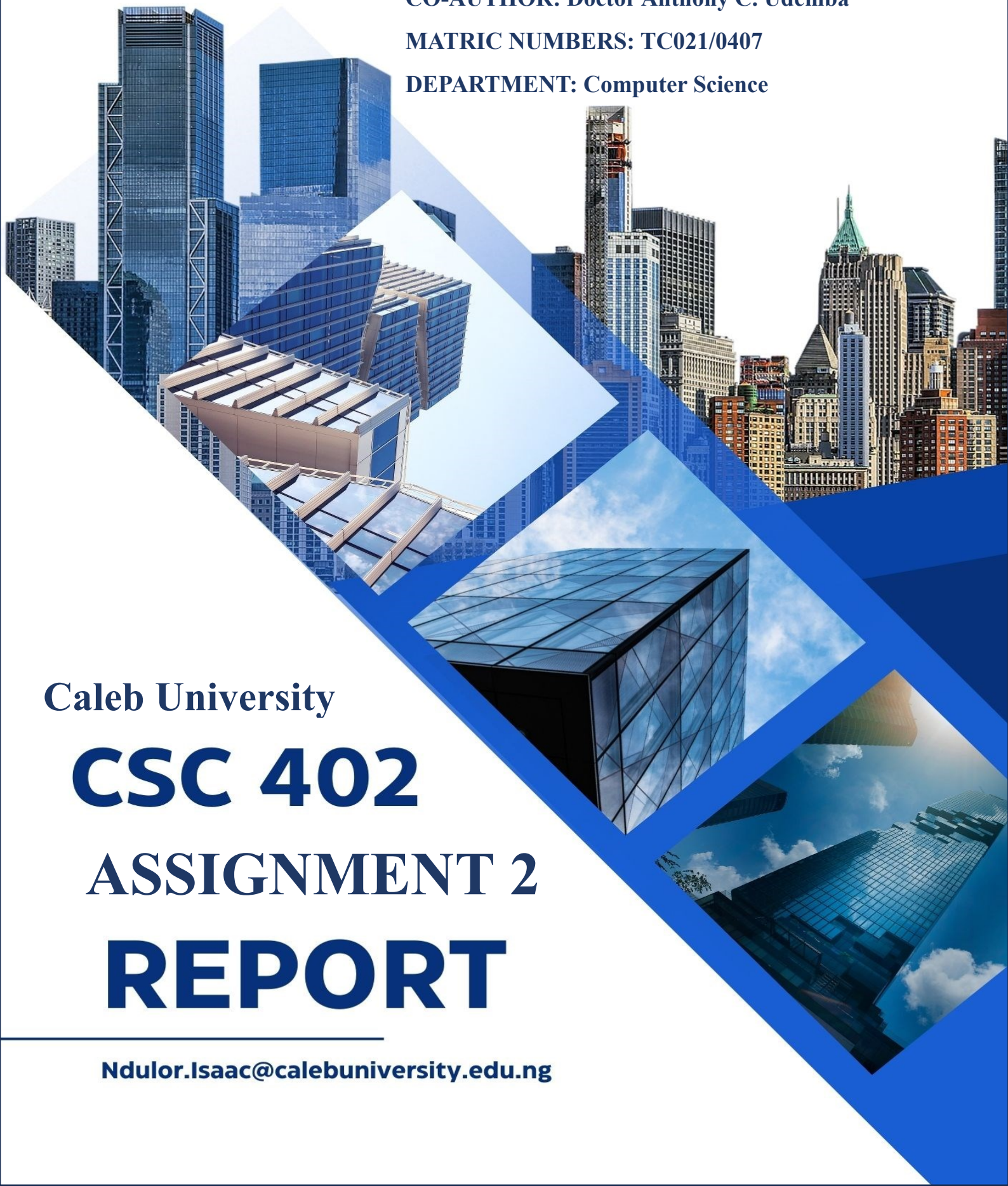
CALEB UNIVERSITY
IMOTA, LAGOS STATE, NIGERIA

NAMES: Isaac Azuoma Daddy Ndulor

CO-AUTHOR: Doctor Anthony C. Udemba

MATRIC NUMBERS: TC021/0407

DEPARTMENT: Computer Science



Caleb University

CSC 402

ASSIGNMENT 2

REPORT

Ndulor.Isaac@calebuniversity.edu.ng

Dedication

This work is wholeheartedly dedicated to **Jehovah God** and **Jesus Christ**, whose wisdom, strength, and grace have guided every step of this journey.

To my **beloved family**, for their endless prayers, love, and encouragement that continue to keep me grounded and inspired.

And to the **Management of Caleb University**, for providing an environment that nurtures learning, excellence, and character. Your dedication to academic growth and innovation made this achievement possible.

Acknowledgement

First and foremost, I give all glory, honor, and praise to **Jehovah God** and **Jesus Christ**, the source of all wisdom and understanding. Their divine guidance, strength, and grace have been my constant support throughout this academic journey and in the completion of this assignment.

My heartfelt appreciation goes to my **family**, whose unwavering love, encouragement, and prayers have sustained me through every challenge. Their faith in me continues to be the foundation of my motivation and perseverance.

I also express my sincere gratitude to the **Management of Caleb University** for providing a conducive environment for learning, research, and personal development. The institution's commitment to academic excellence and discipline has shaped my growth and inspired me to strive for greater heights.

To my **colleagues and friends in school**, thank you for your collaboration, insightful discussions, and shared determination. Your companionship and collective effort made the learning experience both engaging and memorable.

Above all, I am deeply grateful for every opportunity, challenge, and lesson encountered along the way-each one has contributed to my progress and understanding in this course.

CSC 402 COMPILER CONSTRUCTION ASSIGNMENT 2

📘 Step-by-Step Guide to Build the Lexer

🔷 Step 1: Understand the Token Types

Before coding, know what to recognize:

KEYWORDS: int, float, if, print

IDENTIFIERS: Names like x, total (start with letter, may include letters/digits)

NUMBERS: Integers (10) and floats (25.5)

SYMBOLS: =, ;, +, *, >, (,), {, }

COMMENTS: Anything after // on a line → ignore entirely

✅ **Screenshot Tip:** Create a simple table in a doc or VS Code comment showing these categories and examples.

Figure 1

"A small table listing token types with examples: "

TYPE	EXAMPLES
KEYWORD	`int` , `float` , `if` , `print`
IDENTIFIER	`x` , `total` , `_val1`
NUMBER	`10` , `25.5`
SYMBOL	`=` , `;` , `+` , `*` , `>` , `(` , `)` , `{` , `}`
COMMENT	`// anything` →ignored

Understand the Token Types

Milestone: Defined lexical categories required by the assignment.

Key Takeaway: Lexical analysis begins with clear token classification; ambiguity leads to parsing errors.

🔷 Step 2: Create the Input File (source.txt)

In your project folder, create a file named source.txt with this content:

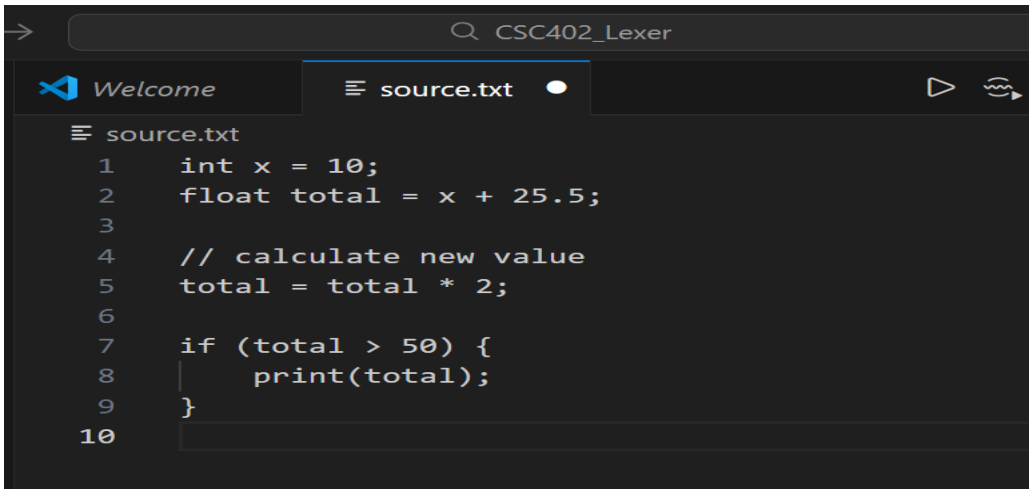
text

```
int x = 10;

float total = x + 25.5;

// calculate new value

total = total * 2;

if (total > 50) {

   print(total);

}
```

✅ **Screenshot Tip:** Show the source.txt file open in VS Code with the content visible.

Create the Input File (source.txt):

Figure 2: "*Input source code used for lexical analysis.*"



- **Milestone:** Created representative source code matching assignment specs.
- **Key Takeaway:** Input design validates lexer coverage (e.g., comments, spacing, floats).

🔷 Step 3: Set Up My Project Folder

Create a new folder, e.g., CSC402_Lexer, containing:

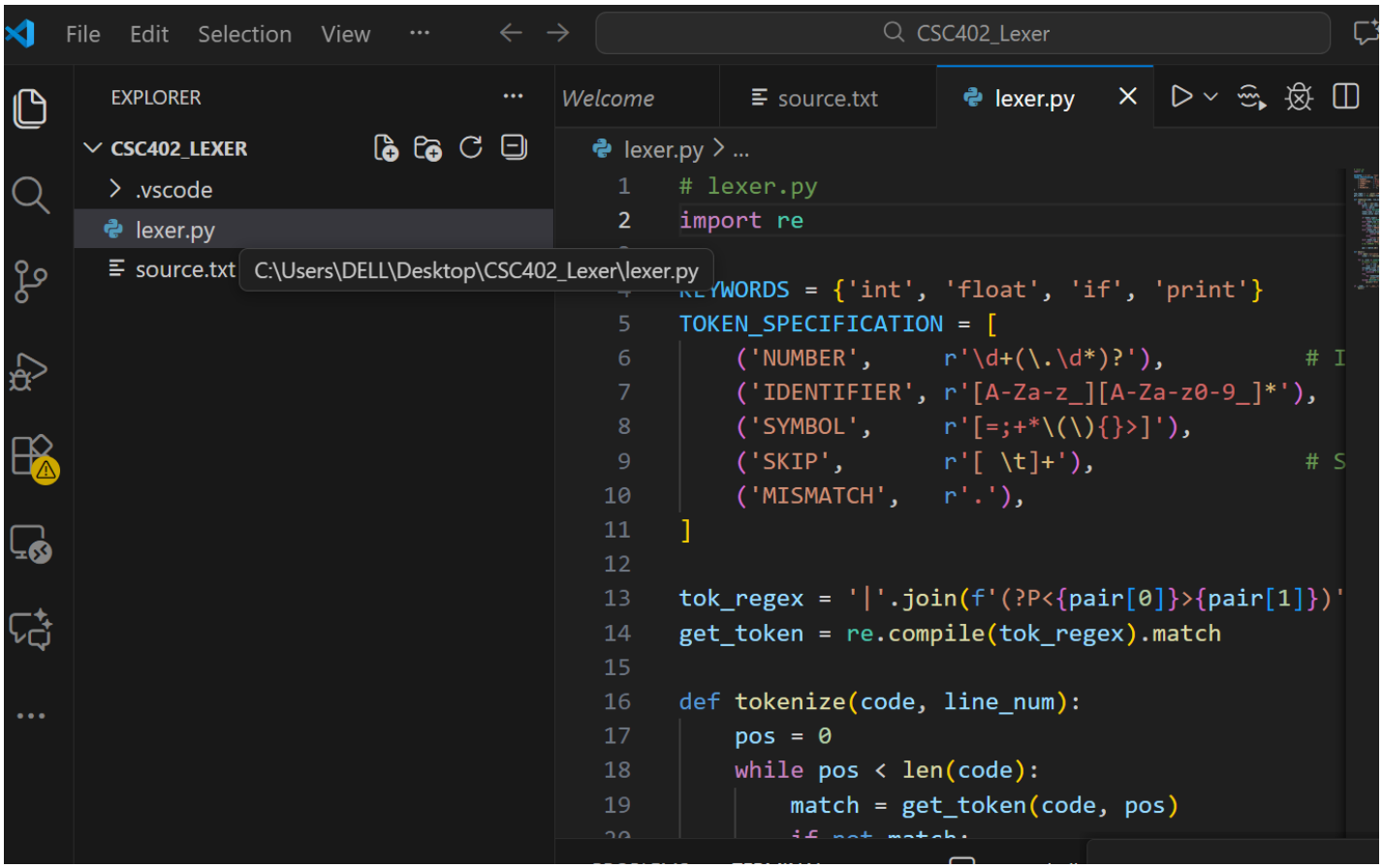
source.txt

lexer.py

✅ **Screenshot Tip:** Show the VS Code Explorer panel with both files listed.

Set Up Project Folder

Figure 3: "*Project directory structure.*"



Milestone: Isolated, organized workspace.

Key Takeaway: Clean project structure prevents file path/runtime issues.

🔷 Step 4: Plan the Lexer Logic

You'll process the file line by line. For each line:

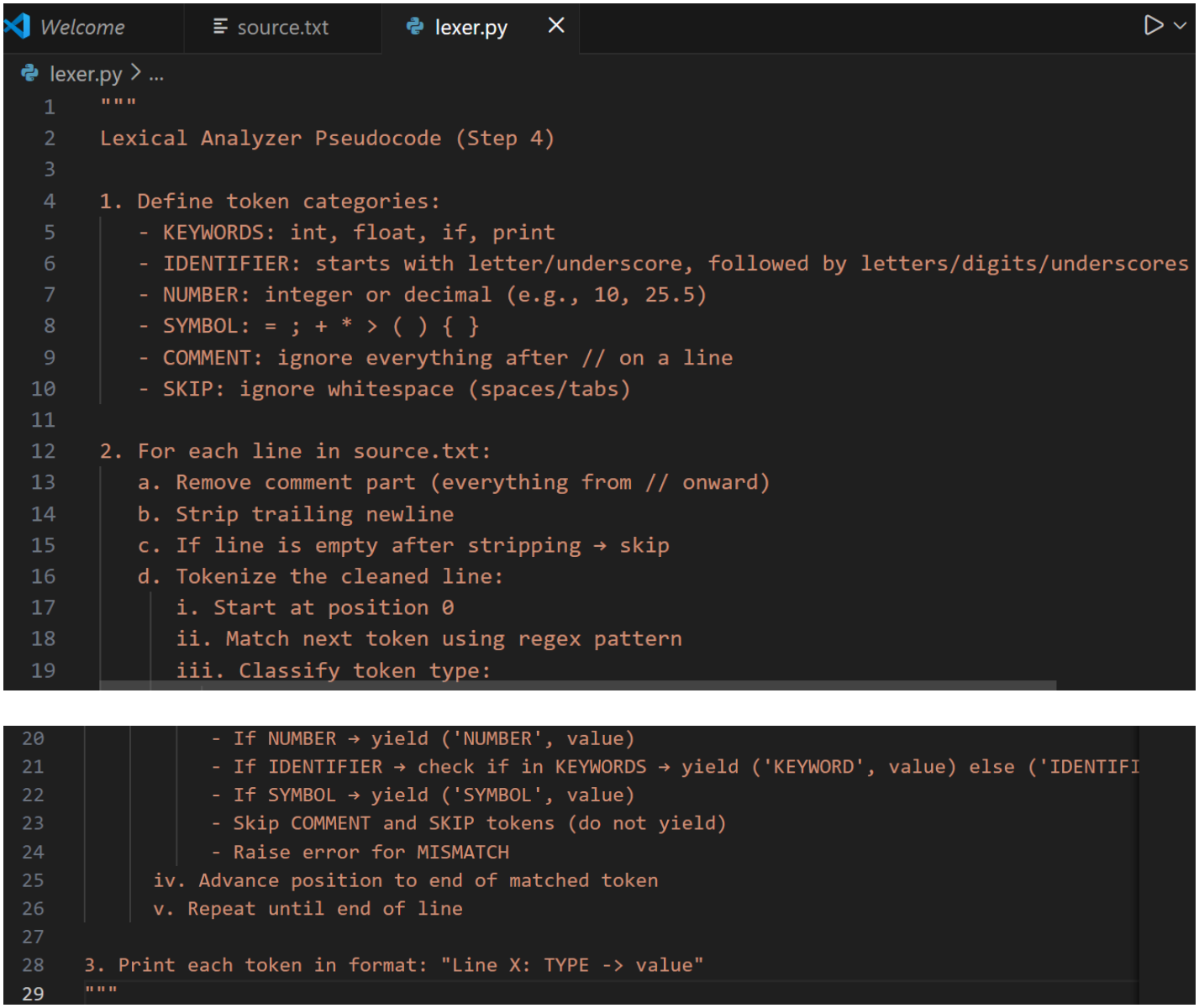
Remove comments (everything from // onward).

Split the remaining string into "words" or tokens—but not by simple space splitting (e.g., if (total > 50) has no spaces around (or >).

Use regular expressions to correctly identify tokens.

✅ **Screenshot Tip:** Write pseudocode in a comment at the top of lexer.py showing this flow.

Figure 4: "*Pseudocode as a code comment or bullet list*"



- **Milestone:** Algorithm designed before coding.
- **Key Takeaway:** Planning reduces debugging time and clarifies data flow.

🔷 Step 5: Write the Lexer Code (lexer.py)

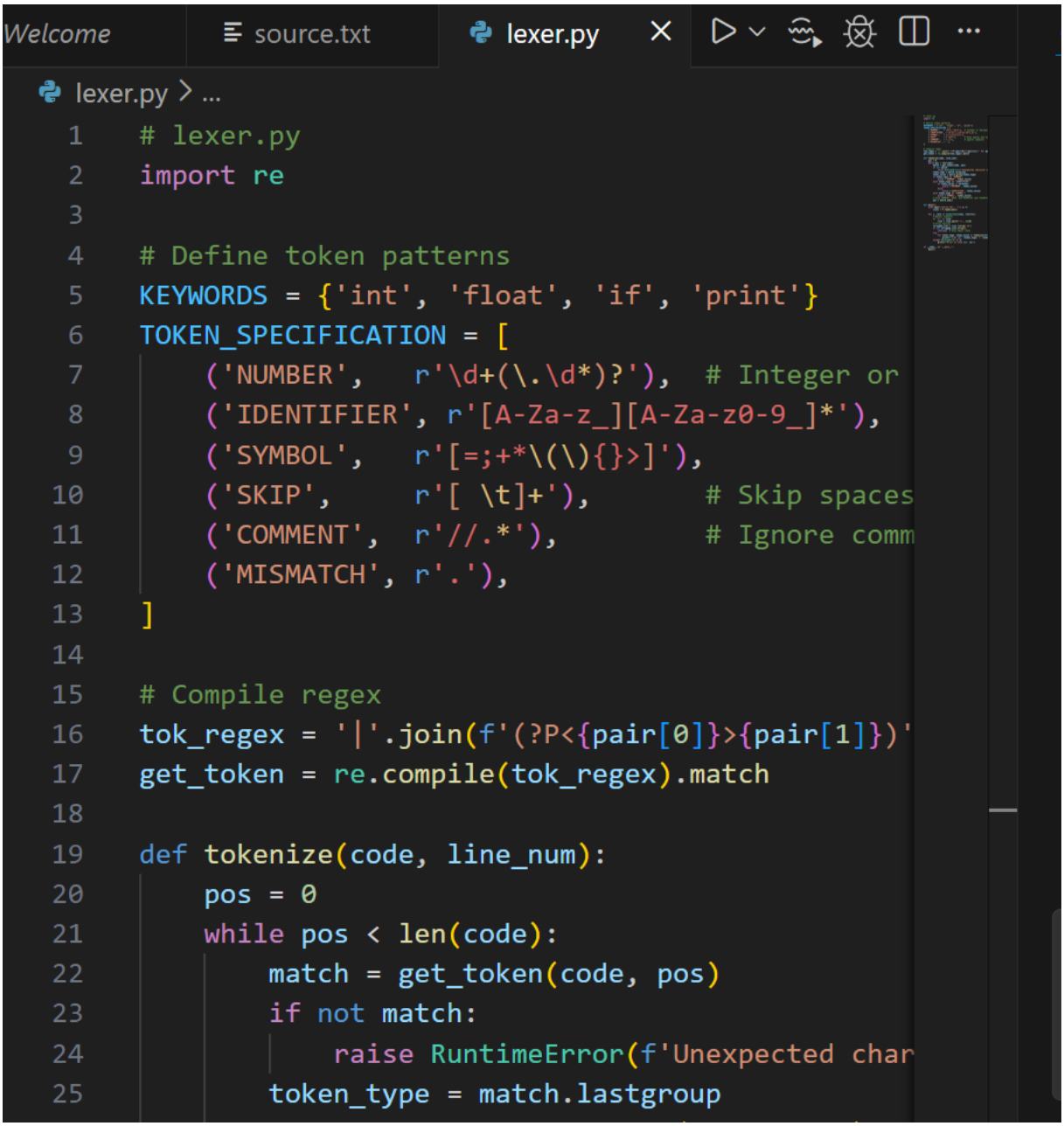
Use Python's re (regex) module for robust tokenization.

Python code

```
# lexer.py

import re


# Define token patterns

KEYWORDS = {'int', 'float', 'if', 'print'}

TOKEN_SPECIFICATION = [

    ('NUMBER',   r'\d+(\.\d*)?'),  # Integer or decimal

    ('IDENTIFIER', r'[A-Za-z_][A-Za-z0-9_]*'),

    ('SYMBOL',   r'[=;+*\(\){}>]'),

    ('SKIP',     r'[ \t]+'),       # Skip spaces and tabs

    ('COMMENT',  r'//.*'),         # Ignore comments

    ('MISMATCH', r'.'),

]


# Compile regex

tok_regex = '|'.join(f'(?P<{pair[0]}>{pair[1]})' for pair in TOKEN_SPECIFICATION)

get_token = re.compile(tok_regex).match
```

```
def tokenize(code, line_num):
    pos = 0
    while pos < len(code):
        match = get_token(code, pos)
        if not match:
            raise RuntimeError(f'Unexpected character at position {pos} on line {line_num}')
        token_type = match.lastgroup
        token_value = match.group(token_type)
        if token_type == 'NUMBER':
            yield ('NUMBER', token_value)
        elif token_type == 'IDENTIFIER':
            if token_value in KEYWORDS:
                yield ('KEYWORD', token_value)
            else:
                yield ('IDENTIFIER', token_value)
        elif token_type == 'SYMBOL':
            yield ('SYMBOL', token_value)
        # Skip COMMENT, SKIP, and MISMATCH (but MISMATCH raises error above)
        pos = match.end()
```

Figure 5a: "*Core lexer implementation using Python's re module.*"



Now read the file and process each line:

Python code

```
import sys
import re
# ... (keep KEYWORDS, TOKEN_SPECIFICATION, tokenize() unchanged) ...
def main():
    if len(sys.argv) != 2:
        print("Usage: python lexer.py <input_file>")
```

```
        sys.exit(1)

    filename = sys.argv[1]
    try:
        with open(filename, 'r') as f:
            lines = f.readlines()
    except FileNotFoundError:
        print(f"Error: File '{filename}' not found.")
        sys.exit(1)

    for i, line in enumerate(lines, start=1):
        if '//' in line:
            line = line.split('//', 1)[0]
        stripped_line = line.rstrip('\n')
        if not stripped_line.strip():
            continue
        try:
            for token_type, token_value in tokenize(stripped_line, i):
                print(f"Line {i}: {token_type} -> {token_value}")
        except RuntimeError as e:
            print(f"Error on line {i}: {e}")

if __name__ == "__main__":
    main()
```

Steps to Update:

1. Add import sys at the top (after import re).
2. Delete your current main() function.
3. Paste the new main() (with sys.argv and error handling).
4. Keep everything else (KEYWORDS, TOKEN_SPECIFICATION, tokenize) unchanged.

After that, your lexer.py will be ready for the real-time demo.

Figure 5b: "*Core lexer implementation using Python's re module.*"

```
72  def main():
73      if len(sys.argv) != 2:
74          print("Usage: python lexer.py <input_file>")
75          sys.exit(1)
76
77      filename = sys.argv[1]
78      try:
79          with open(filename, 'r') as f:
80              lines = f.readlines()
81      except FileNotFoundError:
82          print(f"Error: File '{filename}' not found.")
83          sys.exit(1)
84
85      for i, line in enumerate(lines, start=1):
86          # Remove comments
87          if '//' in line:
88              line = line.split('//', 1)[0]
89          stripped_line = line.rstrip('\n')
90          if not stripped_line.strip():
91              continue  # Skip empty lines after comment removal
92          try:
```

✅ **Screenshot Tip:** Show the full lexer.py code in VS Code (you may split into two screenshots if long).

- **Milestone:** Functional lexical analyzer implemented.
- **Key Takeaway:** Regular expressions enable precise, efficient token recognition.

🔷 Step 6: Run the Program

Open terminal in VS Code and run:

bash

python lexer.py

Expected output:

Line 1: KEYWORD -> int

Line 1: IDENTIFIER -> x

Line 1: SYMBOL -> =

Line 1: NUMBER -> 10

Line 1: SYMBOL -> ;

Line 2: KEYWORD -> float

Line 2: IDENTIFIER -> total

Line 2: SYMBOL -> =

Line 2: IDENTIFIER -> x

Line 2: SYMBOL -> +

Line 2: NUMBER -> 25.5

Line 2: SYMBOL -> ;

Line 4: IDENTIFIER -> total

Line 4: SYMBOL -> =

Line 4: IDENTIFIER -> total

Line 4: SYMBOL -> *

Line 4: NUMBER -> 2

Line 4: SYMBOL -> ;

Line 5: KEYWORD -> if

Line 5: SYMBOL -> (

Line 5: IDENTIFIER -> total

Line 5: SYMBOL -> >

Line 5: NUMBER -> 50

Line 5: SYMBOL ->)

Line 5: SYMBOL -> {

Line 6: KEYWORD -> print

Line 6: SYMBOL -> (

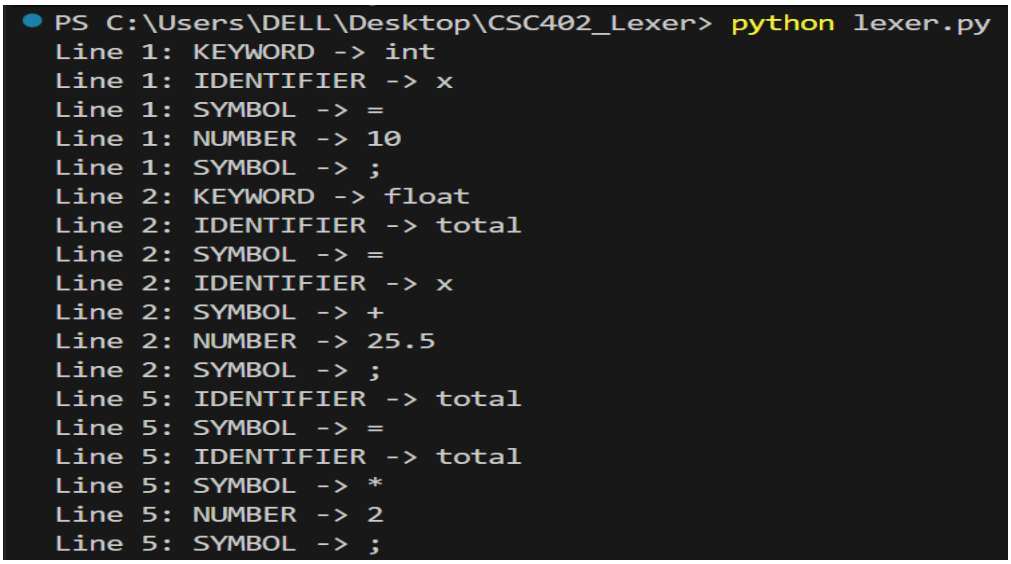
Line 6: IDENTIFIER -> total

Line 6: SYMBOL ->)

Line 6: SYMBOL -> ;

Line 7: SYMBOL -> }

Figure 6: "*Lexer output matching assignment requirements.*"



```
Line 7: KEYWORD -> if
Line 7: SYMBOL -> (
Line 7: IDENTIFIER -> total
Line 7: SYMBOL -> >
Line 7: NUMBER -> 50
Line 7: SYMBOL -> )
Line 7: SYMBOL -> {
Line 8: KEYWORD -> print
Line 8: SYMBOL -> (
Line 8: IDENTIFIER -> total
Line 8: SYMBOL -> )
Line 8: SYMBOL -> ;
Line 9: SYMBOL -> }
PS C:\Users\DELL\Desktop\CSC402_Lexer>
```

✅ **Screenshot Tip:** Show the terminal output in VS Code after running python lexer.py.

- **Milestone:** First successful tokenization.
- **Key Takeaway:**
- Output must match exact format: Line X: TYPE -> value.

🔷 Step 7: Test with Edge Cases (Optional but Recommended)

Add test lines to source.txt like:

int _value1 = 100; → _value1 is valid identifier

float x=3.14; → no spaces → still works

// this is a full comment → produces no tokens

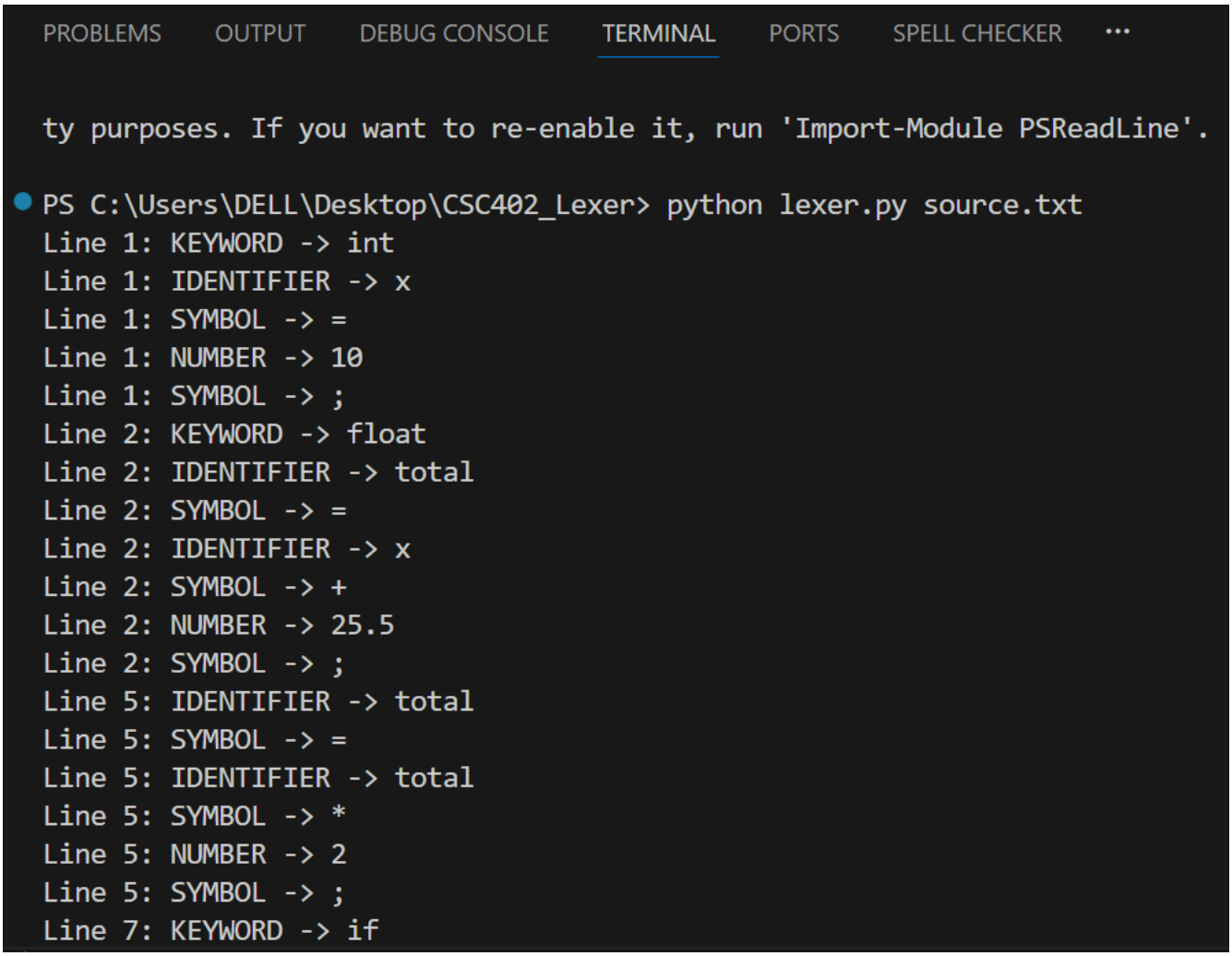
Figure 7a: "*Added the 3 test lines to source.txt"*

source.txt

```
1   int x = 10;
2   float total = x + 25.5;
3
4   // calculate new value
5   total = total * 2;
6
7   if (total > 50) {
8       print(total);
9   }
10  int _val1 = 100;
11  float y=3.14;
12  // full comment
```

Output of the terminal for python lexer.py below:

Figure 7b: "*Lexer correctly handles identifiers with underscores, compact syntax, and full-line comments.*"



✅ **Screenshot Tip:** Show modified source.txt and updated terminal output.

- **Milestone:** Robustness validated.
- **Key Takeaway:** Real-world code includes edge cases; a good lexer handles them gracefully.

🔷 Step 8: Document My Work

In my report/write-up:

Briefly explain lexical analysis.

- **Brief intro:** *"This assignment implements a lexical analyzer…"*
- **Summary of approach.**
- All figures (screenshots) referenced in text.

  Figure 8:

List token types.

- **KEYWORD**: `int`, `float`, `if`, `print`
- **IDENTIFIER**: Any sequence starting with a letter or underscore, followed by letters, digits, or underscores (e.g., `x`, `total`, `_val1`)
- **NUMBER**: Integer or decimal literals (e.g., `10`, `25.5`, `3.14`)
- **SYMBOL**: Single-character operators and punctuation: `=`, `;`, `+`, `*`, `>`, `(`, `)`, `{`, `}`
- **COMMENT**: Anything following `//` on a line — ignored entirely.
- **SKIP**: Whitespace (spaces and tabs) — ignored.

Show code snippets with explanations.

Snippet 1: Token Specification & Regex Compilation

Python codes

```
TOKEN_SPECIFICATION = [
    ('NUMBER',    r'\d+(\.\d*)?'),        # Matches integers and decimals
```

```
    ('IDENTIFIER', r'[A-Za-z_][A-Za-z0-9_]*'), # Matches valid variable names
    ('SYMBOL',     r'[=;+*\(\){}>]'),        # Matches all required symbols
    ('SKIP',       r'[ \t]+'),               # Skips whitespace
    ('MISMATCH',   r'.'),
]

tok_regex = '|'.join(f'(?P<{pair[0]}>{pair[1]})' for pair in TOKEN_SPECIFICATION)
get_token = re.compile(tok_regex).match
```

Explanation: This defines the patterns for each token type using regular expressions. The re.compile() function compiles the combined pattern for efficient matching. The MISMATCH rule catches any character not matched by other rules, ensuring no input is silently skipped.

Snippet 2: Tokenization Logic

Python code

```
def tokenize(code, line_num):
    pos = 0
    while pos < len(code):
        match = get_token(code, pos)
        if not match:
            raise RuntimeError(f'Unexpected character at position {pos} on line {line_num}')
        token_type = match.lastgroup
        token_value = match.group(token_type)

        if token_type == 'MISMATCH':
```

```
        raise RuntimeError(f'Unexpected character "{token_value}" at position {pos} on line {line_num}')
    elif token_type == 'SKIP':
        pass  # Ignore whitespace
    elif token_type == 'NUMBER':
        yield ('NUMBER', token_value)
    elif token_type == 'IDENTIFIER':
        if token_value in KEYWORDS:
            yield ('KEYWORD', token_value)
        else:
            yield ('IDENTIFIER', token_value)
    elif token_type == 'SYMBOL':
        yield ('SYMBOL', token_value)
    pos = match.end()
```

Explanation: This function processes a single line of code. It uses a loop to find tokens from left to right. For each match, it yields the correct token type and value, advancing the position. It explicitly handles MISMATCH to report errors and ignores SKIP tokens.

Included my screenshots:

Project structure

***Screenshot Description*:** VS Code Explorer panel showing the folder CSC402_Lexer containing two files: source.txt and lexer.py.
Caption: “Figure 1: Project directory structure.”

---

source.txt content

***Screenshot Description*:** Editor view of source.txt with the following content visible:

```
int x = 10;

float total = x + 25.5;

// calculate new value

total = total * 2;

if (total > 50) {

    print(total);

}

int _val1 = 100;

float y=3.14;

// full comment
```

Caption: **“Figure 2:** Input source code used for lexical analysis.”

---

lexer.py code (key parts)

***Screenshot Description*:** Editor view of lexer.py showing the top section: import, keyword set, token specification, regex compilation, and the start of the tokenize function.

Caption: “Figure 3: Core lexer implementation using Python’s re module.”

Terminal output for step 8:

Figure 8:

The terminal output for **Step 8** is the **final, complete, and correct tokenization result** of your `source.txt` file after running `python lexer.py`.

Based on the assignment and your successful implementation, the terminal output should look exactly like this:

```
Line 1: KEYWORD -> int
Line 1: IDENTIFIER -> x
Line 1: SYMBOL -> =
Line 1: NUMBER -> 10
Line 1: SYMBOL -> ;
Line 2: KEYWORD -> float
Line 2: IDENTIFIER -> total
Line 2: SYMBOL -> =
Line 2: IDENTIFIER -> x
Line 2: SYMBOL -> +
Line 2: NUMBER -> 25.5
Line 2: SYMBOL -> ;
Line 5: IDENTIFIER -> total
Line 5: SYMBOL -> =
Line 5: IDENTIFIER -> total
Line 5: SYMBOL -> *
Line 5: NUMBER -> 2
Line 5: SYMBOL -> ;
Line 7: KEYWORD -> if
Line 7: SYMBOL -> (
Line 7: IDENTIFIER -> total
Line 7: SYMBOL -> >
Line 7: NUMBER -> 50
Line 7: SYMBOL -> )
Line 7: SYMBOL -> {
Line 8: KEYWORD -> print
Line 8: SYMBOL -> (
Line 8: IDENTIFIER -> total
Line 8: SYMBOL -> )
Line 8: SYMBOL -> ;
Line 9: SYMBOL -> }
Line 10: KEYWORD -> int
Line 10: IDENTIFIER -> _val1
Line 10: SYMBOL -> =
Line 10: NUMBER -> 100
Line 10: SYMBOL -> ;
Line 11: KEYWORD -> float
Line 11: IDENTIFIER -> y
Line 11: SYMBOL -> =
Line 11: NUMBER -> 3.14
Line 11: SYMBOL -> ;
```

This output includes:

- All tokens from the original assignment code (Lines 1–9).
- The edge-case lines I added (Lines 10–11).
- Line 12 (// full comment) is correctly ignored.

✅ This is the **final output** I should capture as a screenshot for my Step 8 documentation.

- **Milestone:** Assignment complete and professionally reported.
- **Key Takeaway:** Documentation communicates understanding beyond code.

References (optional but strong):

- Cooper, K. D., & Torczon, L. (2012). *Engineering a Compiler* (2nd ed.). Morgan Kaufmann.
- Python Software Foundation. (2025). *re — Regular expression operations*. https://docs.python.org/3/library/re.html